High-Level Design (HLD)

RAG-Based Customer Support Assistant

Version: 1.0
Date: April 2026
Author: Moin Uddin

RAG-Based Customer Support Assistant
with LangGraph & Human-in-the-Loop Escalation

Table of Contents

1. System Overview
2. Architecture Diagram
3. Component Description
4. Data Flow
5. Technology Choices
6. Scalability Considerations

1. System Overview

1.1 Problem Definition

Traditional customer support systems face several critical challenges:

- **High Volume:** Support teams are overwhelmed with repetitive queries that could be automated.
- **Inconsistency:** Different agents provide varying quality of answers to the same questions.
- **Latency:** Customers wait in queues for simple answers that exist in documentation.
- **Knowledge Gaps:** New agents lack institutional knowledge, leading to incorrect or incomplete responses.
- **Scalability:** Hiring and training human agents is expensive and slow.

The fundamental problem is: **How can we deliver instant, accurate, and contextually relevant customer support at scale while maintaining the ability to escalate complex or sensitive cases to human agents?**

1.2 Scope of the System

This system is a **Retrieval-Augmented Generation (RAG)** customer support assistant that:

In Scope	Out of Scope
PDF-based knowledge base ingestion	Multi-language support
Semantic search with vector embeddings	Real-time knowledge base updates
LLM-powered answer generation	Payment processing or account modifications
Intent-based query classification & routing	Integration with external CRM systems
Human-in-the-Loop (HITL) escalation	Multi-modal input (images, voice)
Web-based chat interface (Streamlit)	Mobile application

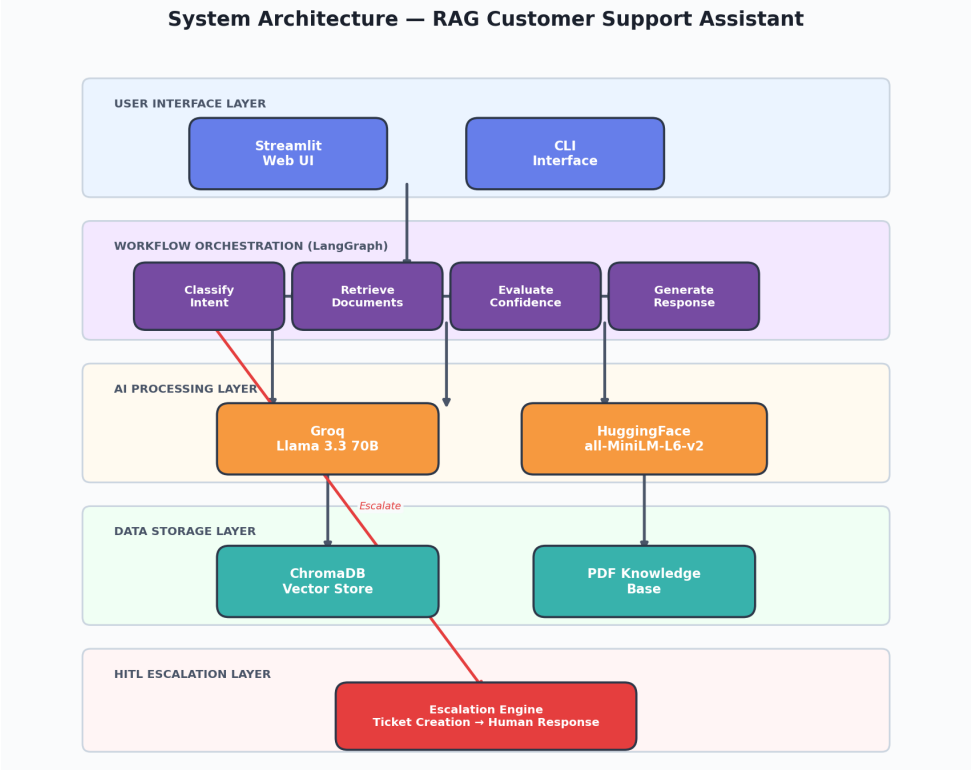
Conversation history within a session	Cross-session memory persistence

1.3 Key Design Principles

1. **Accuracy over Speed:** The system prioritizes correct answers over fast but wrong ones. When uncertain, it escalates.
2. **Transparency:** Every response shows the source documents used, building user trust.
3. **Graceful Degradation:** When components fail (LLM errors, empty retrieval), the system falls back to human escalation rather than crashing.
4. **Modularity:** Each component is independently replaceable (swap ChromaDB for Pinecone, swap Gemini for GPT-4, etc.).

---

2. Architecture Diagram



2.1 System Architecture

```
■■■
```

```
■                        ▼                                          ■
■■■
```

```
          ■■■■■■■■■■■■■■■■■■■■
                   ■
        ■■■■■■■■■■■■■■■■■■■■■■■■
        ■          ■          ■
[Escalate]  [Normal]  [Out of Scope]
        ■          ■          ■
        ■          ▼          ■
        ■  ■■■■■■■■■■■■■■■■■  ■
        ■  ■  ChromaDB     ■  ■
        ■  ■  Retriever    ■  ■
        ■  ■■■■■■■■■■■■■■■■■  ■
        ■          ■          ■
        ■   ■■■■■■■■■■■■■■  ■
        ■   ■ Confidence ■  ■
        ■   ■ Evaluator  ■  ■
        ■   ■■■■■■■■■■■■■■  ■
        ■     ■■■■■■■■■■    ■
        ■  [High]  [Low]    ■
        ■     ■       ■      ■
        ■     ▼       ■      ■
        ■  ■■■■■■■■■  ■      ■
        ■  ■ LLM    ■  ■      ■
        ■  ■ Gen.   ■  ■      ■
        ■  ■■■■■■■■■  ■      ■
        ■     ■       ■      ■
        ■     ■       ▼      ■
        ■     ■  ■■■■■■■■■■■■■
        ■■■■■■■■■■■  HITL  ■■
              ■  ■■■■■■■■■■■■■
              ■       ■      ■
              ▼       ▼      ■
          ■■■■■■■■■■■■■■■■■■■ ■
          ■   Response    ■■■
          ■■■■■■■■■■■■■■■■■■
```

---

3. Component Description

3.1 Document Loader

- **Purpose:** Extracts text from PDF documents and converts them into structured document objects.
- **Technology:** PyPDFLoader (LangChain Community)
- **Input:** PDF file path
- **Output:** List of Document objects with page content and metadata (page number, source file)
- **Design Decision:** PyPDFLoader was chosen over alternatives (e.g., UnstructuredPDFLoader) for its simplicity, speed, and reliability with standard PDF formats. It processes page-by-page, preserving document structure.

3.2 Chunking Strategy

- **Purpose:** Splits large documents into smaller, semantically coherent segments suitable for embedding.
- **Technology:** RecursiveCharacterTextSplitter
- **Parameters:**

• Chunk Size: 500 characters (optimal balance between context and precision)

• Chunk Overlap: 100 characters (20% overlap to prevent context loss at boundaries)

• Separators: `["\n\n", "\n", ". ", " ", ""]` (priority order)

• **Design Decision:** Recursive splitting was chosen because it tries to split on natural text boundaries (paragraphs first, then sentences, then words), preserving semantic meaning better than fixed-size splitting. The 500-character chunk size provides enough context for answering specific questions while keeping embeddings focused.

3.3 Embedding Model

• **Purpose:** Converts text chunks and queries into high-dimensional vector representations for semantic search.

• **Technology:** HuggingFace Embeddings (`all-MiniLM-L6-v2`) — runs locally

• **Output Dimension:** 384-dimensional vectors

• **Design Decision:** A local HuggingFace model was chosen for its:

• Zero API cost and no rate limits (runs entirely on your machine)

• No external API dependency (works offline, no network latency)

• Fast inference with small model footprint (~80MB cached locally)

• High quality sentence embeddings well-suited for semantic search

3.4 Vector Store

• **Purpose:** Stores and indexes document embeddings for efficient similarity search.

• **Technology:** ChromaDB (persistent mode)

• **Storage:** Local filesystem (`./chroma_db/`)

• **Search Method:** Cosine similarity

• **Design Decision:** ChromaDB was selected because:

• Lightweight and self-contained (no external server needed)

• Supports persistent storage (survives restarts)

• Open-source with active development

• Built-in support for metadata filtering

• Easy integration with LangChain

3.5 Retriever

• **Purpose:** Finds the most relevant document chunks for a given query.

• **Configuration:** Top-K = 3 (returns 3 most similar chunks)

• **Features:**

• Similarity search with relevance scores (0.0 to 1.0)

• Confidence evaluation for HITL escalation decisions

• Context formatting with source attribution

• **Design Decision:** K=3 was chosen to provide sufficient context without overwhelming the LLM prompt. Scores are used for confidence-based routing.

3.6 LLM

• **Purpose:** Generates natural language responses using retrieved context and handles intent classification.

• **Technology:** Groq (Llama 3.3 70B Versatile)

• **Temperature:** 0.3 (low for factual, consistent responses)

• **Design Decision:** Groq with Llama 3.3 70B was chosen for:

• Generous free tier (14,400 requests/day, no credit card required)

• Ultra-fast inference via Groq's custom LPU (Language Processing Unit) hardware

• Llama 3.3 70B offers excellent reasoning and factual accuracy

• Support for system prompts and structured outputs

3.7 Graph Workflow Engine

• **Purpose:** Orchestrates the entire query processing pipeline with conditional routing.

• **Technology:** LangGraph StateGraph

• **Nodes:** 5 processing nodes (classify, retrieve, evaluate, generate, escalate)

• **Edges:** 2 conditional edges (intent-based, confidence-based) + 2 direct edges

• **State:** TypedDict with 12 fields tracking query lifecycle

• **Design Decision:** LangGraph was chosen over simple sequential chains because:

• Supports conditional branching (essential for escalation routing)

• Provides explicit state management

• Enables graph visualization for debugging

• Built-in support for HITL interrupts

3.8 Routing Layer

• **Purpose:** Makes dynamic decisions about query handling based on intent and confidence.

• **Two Decision Points:**

1. **Post-Intent:** Routes COMPLAINT/ESCALATE/OUT_OF_SCOPE directly to HITL

2. **Post-Confidence:** Routes low-confidence retrievals to HITL

• **Design Decision:** Two-stage routing ensures that obviously escalation-worthy queries skip unnecessary retrieval, while ambiguous queries get a chance at automated resolution before escalation.

3.9 HITL Module

• **Purpose:** Handles the transition from automated to human-assisted support.

• **Features:**

• Escalation trigger evaluation

• Context-rich ticket creation

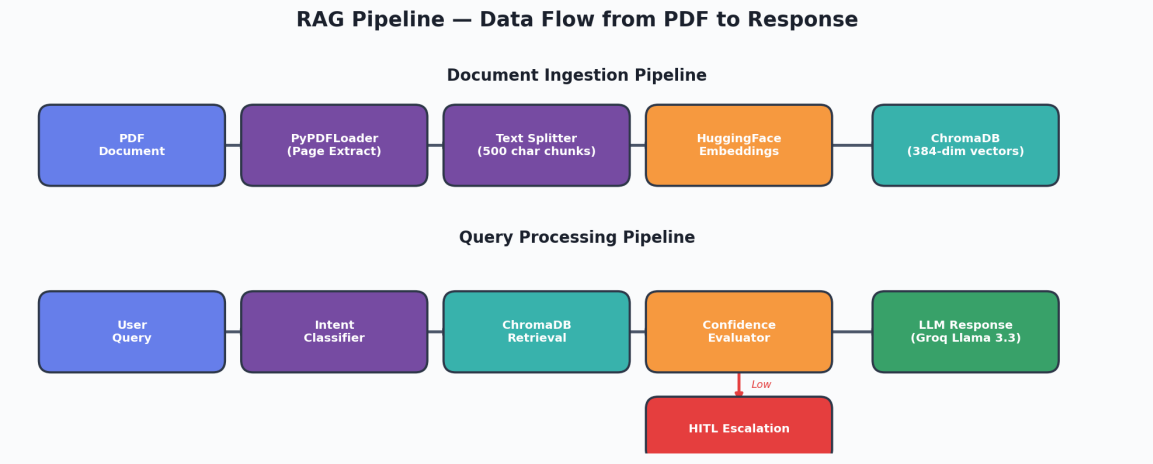
• Human response integration

• User-facing escalation messages

• **Design Decision:** The HITL module is designed as a simulation within the Streamlit UI, where a "human agent" can type a response. In production, this would connect to a ticketing system (e.g., Zendesk, Freshdesk).

---

4. Data Flow



4.1 Document Ingestion Pipeline (PDF → Vector Store)

```
PDF File → PyPDFLoader → [Document Objects per page]
                                ■
                                ▼
                      RecursiveCharacterTextSplitter
                                ■
                      [Chunked Documents with Metadata]
                                ■
                                ▼
                      HuggingFace Embedding Model (local)
                                ■
                      [384-dim Vector Embeddings]
                                ■
                                ▼
                      ChromaDB (Persistent Storage)
                      - Vectors + Metadata + Raw Text
```

Data Transformations:

1. **PDF → Pages:** Binary PDF → List of text strings (one per page)
2. **Pages → Chunks:** Long text → 500-char segments with 100-char overlap
3. **Chunks → Embeddings:** Text strings → 384-dimensional float vectors
4. **Embeddings → ChromaDB:** Vectors stored with metadata (source, page, chunk_index)

4.2 Query Lifecycle (User Query → Answer)

```
User Query (string)
    ■
    ▼
[1] Intent Classification (LLM)
    ■   Output: intent category + escalation flag
```

```
    ■
    ■■■ [ESCALATION PATH] ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    ■                                                   ■
    ▼                                                   ■
[2] Document Retrieval (ChromaDB)                       ■
    ■   Output: top-3 docs with scores                  ■
    ■                                                   ■
    ▼                                                   ■
[3] Confidence Evaluation                               ■
    ■   Output: confidence level + escalation flag       ■
    ■                                                   ■
    ■■■ [LOW CONFIDENCE] ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    ■                                                   ■
    ▼                                                   ▼
[4] Response Generation (LLM)               [5] HITL Escalation
    ■   Input: query + context                  ■   Output: ticket + message
    ■   Output: answer string                   ■
    ■                                           ■
    ▼                                           ▼
[6] Response returned to user with sources/escalation info
```

---

5. Technology Choices

5.1 Why ChromaDB?

Criterion	ChromaDB	Alternatives (Pinecone, Weaviate)
Setup	Zero config, local file	Requires cloud account or Docker
Cost	Free, open-source	Free tiers limited, then paid
Persistence	Built-in file persistence	Varies
LangChain Integration	First-class support	Supported but more setup
Learning Curve	Minimal	Moderate
Production Ready	Good for small-medium scale	Better for large scale

Verdict: ChromaDB is ideal for this project's scope — a demonstration system with moderate document volume. Its zero-configuration local setup eliminates deployment complexity.

5.2 Why LangGraph?

Criterion	LangGraph	Alternatives (LangChain LCEL, Custom)
Conditional Routing	Native support	Requires custom logic
State Management	Built-in TypedDict state	Manual implementation
HITL Support	interrupt() function	No built-in support

Visualization	Graph visualization	No visualization
Debugging	Clear node-by-node execution	Opaque chain execution
Complexity	Moderate	Simple (LCEL) / High (Custom)

Verdict: LangGraph is the natural choice for workflow-based RAG systems. Its conditional edges map directly to our intent-based and confidence-based routing requirements, and its HITL support via `interrupt()` is production-grade.

5.3 LLM Choice: Groq (Llama 3.3 70B Versatile)

- **Why:** Free tier (14,400 req/day), ultra-fast inference via Groq LPU hardware, excellent 70B reasoning
- **Temperature 0.3:** Prioritizes consistent, factual responses over creative ones
- **Alternative considered:** OpenAI GPT-4o (better quality but costly), Google Gemini Flash (similar tier but slower inference)
- **Future:** The modular design allows swapping to any LangChain-supported LLM

5.4 Additional Tools

- **Streamlit:** Rapid UI prototyping with built-in chat components
- **PyPDF:** Reliable PDF text extraction
- **python-dotenv:** Secure environment variable management

---

6. Scalability Considerations

6.1 Handling Large Documents

Challenge	Current Approach	Scaling Strategy
Large PDFs (100+ pages)	Single-threaded processing	Batch processing with multiprocessing
Multiple PDFs	Sequential ingestion	Parallel ingestion pipeline
Chunk volume (10,000+)	ChromaDB local	Migrate to Pinecone/Weaviate cloud
Storage growth	Local filesystem	Cloud object storage (S3)

6.2 Increasing Query Load

Challenge	Current Approach	Scaling Strategy

Concurrent users	Single Streamlit instance	Deploy behind load balancer (multiple instances)
LLM rate limits	Sequential API calls	Request queuing + rate limiting middleware
Vector search latency	ChromaDB local	Distributed vector DB with replicas
Session management	In-memory (Streamlit)	Redis-backed session store

6.3 Latency Concerns

Operation	Current Latency	Optimization
Intent Classification	~1-2s (LLM call)	Cache common intents; keyword-based pre-filter
Vector Search	~50-100ms	Quantized embeddings; approximate nearest neighbors
LLM Generation	~2-4s	Streaming responses; model distillation
Total E2E	~4-8s	Pipeline parallelization; pre-computed embeddings

6.4 Future Architecture Considerations

For production deployment at scale:

1. **Microservices:** Split into ingestion service, retrieval service, and generation service
2. **Message Queue:** Use RabbitMQ/Kafka for async processing between services
3. **Caching Layer:** Redis cache for frequent queries and intent classifications
4. **Monitoring:** LangSmith tracing for production observability
5. **CDN:** Serve the web UI from a CDN for global access
6. **Auto-scaling:** Kubernetes-based deployment with horizontal pod autoscaling

---

End of High-Level Design Document